

Assignment #4: Internship and Job Tracking Dashboard

Course: Tools & Techniques for Data Science

Team Members	
Names	Roll Numbers
Rana M. Zain Zahid	L1F23BSDS0013
Usman Butt	L1F23BSDS0006
Muhammad Abdullah	L1F23BSDS0083

1. Introduction

This report documents the development and deployment of a containerized Internship and Job Tracking Dashboard. Built using Streamlit for the frontend, PostgreSQL for persistent data storage, and Docker Compose for orchestration, the system provides a robust internal tool for university faculty to track, manage, and analyze student placement opportunities. The project successfully integrates multi-page navigation, role-based access control, and comprehensive data visualizations.

2. Problem Statement

The university department currently receives internship and job records from various companies but lacks a centralized, easily accessible system to manage them. The department requires an internal web application where faculty can efficiently add opportunities, verify stored data, analyze hiring trends, and manage record lifecycles securely through a real database system rather than relying on disparate files.

3. System Architecture

The system follows a modern, containerized architecture utilizing Docker Compose to ensure reproducibility across any environment.



Source code, documentation, and version history are maintained in a shared GitHub Repository .

4. Database Design

The application utilizes a PostgreSQL relational database named `student_opportunities_db`. The primary table, `opportunities`, was designed to capture comprehensive job details with strict data constraints to maintain integrity .

Key schema elements include:

- **Primary Key:** `opportunity_id` (SERIAL)
- **Constraints:** NOT NULL on essential fields (`company_name`, `job_title`, `category`).
- **Checks:** `work_mode` restricted to ('Remote', 'Onsite', 'Hybrid') and `status` restricted to ('Open', 'Closed', 'Expired', 'Shortlisted').
- **Defaults:** Currency defaults to 'PKR' and `created_at` timestamps are automatically generated.
- Seed data containing 40 initial records across various cities and technical domains was injected using an initialization script (`seed_data.sql`).

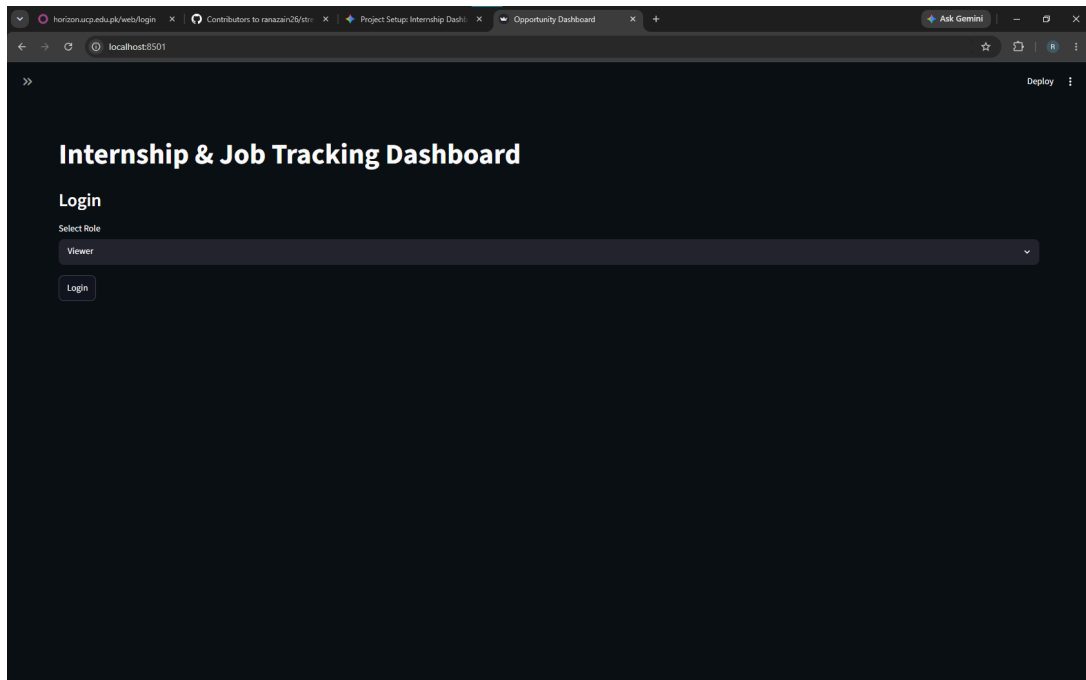
5. Docker Compose Explanation

The environment is orchestrated using `docker-compose.yml`, which defines three interdependent services:

1. **postgres_db:** Runs the `postgres:16` image. It maps port 5433 on the host to 5432 internally. It utilizes named volumes (`postgres_data`) to prevent data loss when the container stops.
2. **pgadmin:** A graphical interface for database administration mapped to host port 5050. It uses `depends_on` to ensure the database starts first.
3. **streamlit_app:** The custom frontend built via a Dockerfile. It operates on port 8501 and communicates with the database using internal Docker networking variables (`DB_HOST=postgres_db`).

6. Streamlit Application Screenshots

Home Page:



Add Opportunity Form:

A screenshot of the 'Add New Opportunity' form in the application. The browser tabs include 'horizon.ucp.edu.pk/web/login', 'Contributors to ranain26/st...', 'Project Setup: Internship Dash...', 'Streamlit', and 'Ask Gemini'. The address bar shows 'localhost:8501/Add_Opportunity'. On the left, a sidebar menu lists options: 'main', 'Add Opportunity' (highlighted), 'View Search', 'Update Opportunity', 'Delete Opportunity', 'Analytics Dashboard', 'CSV Upload Export', 'Duplicate Detection', 'Deadline Alerts', and 'Database Health Check'. The main form area has a title '+ Add New Opportunity' with a 'Go' link. The form fields are arranged in two columns: 'Company Name*' (text input), 'Work Mode' (dropdown menu with 'Remote' selected), 'Job Title*' (text input), 'Min Salary' (text input with '50000' and minus/plus buttons), 'Category' (dropdown menu with 'Data Science' selected), 'Max Salary' (text input with '150000' and minus/plus buttons), 'City' (text input), 'Experience Level' (dropdown menu with 'Entry-Level' selected), 'Country' (text input with 'Pakistan' selected), 'Deadline' (text input with '2026/06/11'), 'Required Skills*' (text area), and 'Source Link' (text input). A 'Save Record' button is at the bottom left of the form.

7. CRUD Operations Explanation

The application performs full Create, Read, Update, and Delete operations using SQLAlchemy to execute raw SQL queries.

- **Create:** Validated via st.form; executes INSERT INTO opportunities...

- **Read:** Dynamic SELECT queries driven by sidebar multiselect widgets in the "View & Search" module.
- **Update:** Allows Admins to select a record ID and execute an UPDATE query modifying statuses, deadlines, and salary bands.
- **Delete:** A secure portal for Admins to execute DELETE FROM opportunities WHERE opportunity_id = :id permanently removing the record.

8. CSV Upload and Export Explanation

To handle bulk data entry, the system utilizes the pandas library.

- **Upload:** Admins upload a .csv file. The app reads it into a DataFrame and utilizes the `df.to_sql(if_exists="append")` method to stream the records directly into the PostgreSQL table.
- **Export:** The system executes a broad `SELECT *` query, converts the resulting DataFrame to CSV byte format, and utilizes `st.download_button` to serve the file to the user.

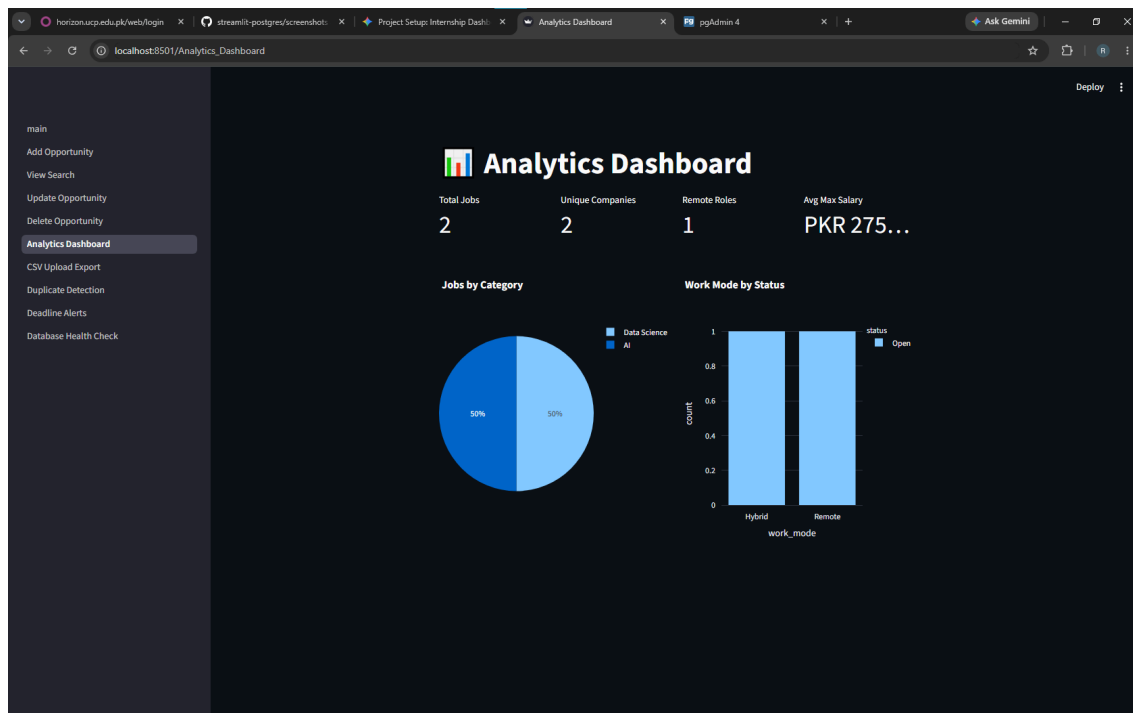
9. Login and Role Management Explanation

Basic session state management (`st.session_state`) was implemented to control data mutation access.

- **Viewers:** Have read-only access to browse jobs, view the dashboard, and check alerts.
- **Admins:** Unlocked via a dropdown selection, granting exclusive access to the Add, Update, Delete, and CSV Upload pages, preventing unauthorized modifications to the database.

10. Dashboard Explanation

The Analytics Dashboard leverages `plotly.express` to visualize the database state dynamically.



The dashboard retrieves all records and calculates high-level KPIs (Total Jobs, Unique Companies, Remote Roles, Avg Max Salary). It includes a pie chart visualizing the distribution of job categories and a bar chart analyzing work modes split by application status.

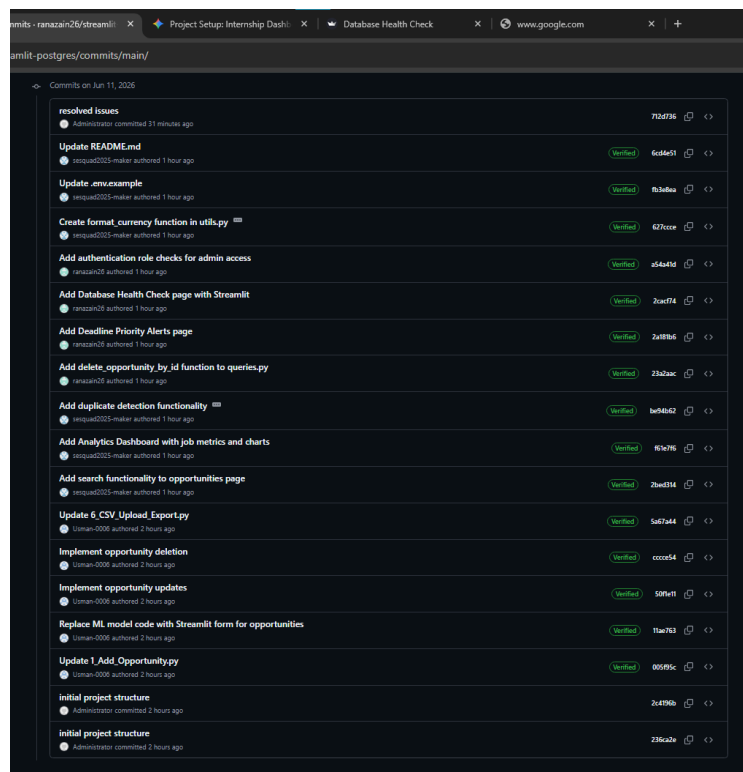
11. GitHub Contribution Evidence

Version control was strictly maintained via GitHub. Group members independently cloned the repository, branched their features, and pushed commits.

Repository Overview:

-

Commit History (Showing all members):



Commits on Jun 11, 2026				
resolved issues				
Administrator committed 31 minutes ago	713d736			
Update README.md				
hoqua0023-maker authored 1 hour ago	6c3de31	Verified		
Update smuzsample				
hoqua0023-maker authored 1 hour ago	8c3af8a	Verified		
Create format_currency function in utils.py				
hoqua0023-maker authored 1 hour ago	627c9e	Verified		
Add authentication role checks for admin access				
ranazai26 authored 1 hour ago	a5d4f4d	Verified		
Add Database Health Check page with Streamlit				
ranazai26 authored 1 hour ago	2c4ef74	Verified		
Add Deadline Priority Alerts page				
ranazai26 authored 1 hour ago	2a188a6	Verified		
Add delete_opportunity_by_id function to queries.py				
ranazai26 authored 1 hour ago	23a2ae	Verified		
Add duplicate detection functionality				
hoqua0023-maker authored 1 hour ago	bef8b42	Verified		
Add Analytics Dashboard with job metrics and charts				
hoqua0023-maker authored 1 hour ago	813e796	Verified		
Add search functionality to opportunities page				
hoqua0023-maker authored 1 hour ago	2ba0314	Verified		
Update 6 CSV Upload Export.py				
Uman-0006 authored 2 hours ago	5a27a44	Verified		
Implement opportunity deletion				
Uman-0006 authored 2 hours ago	ccece54	Verified		
Implement opportunity updates				
Uman-0006 authored 2 hours ago	509ff41	Verified		
Replace ML model code with Streamlit form for opportunities				
Uman-0006 authored 2 hours ago	11ae783	Verified		
Update 1_Add_Opportunity.py				
Uman-0006 authored 2 hours ago	005d95c	Verified		
initial project structure				
Administrator committed 2 hours ago	2c4f96b			
initial project structure				
Administrator committed 2 hours ago	236ca3e			

Contribution Table:

Member Name	Tasks Completed	GitHub Username
Rana M. Zain Zahid	Repo init, Docker Compose, Database Config, Alert Systems	ranazain26
Usman Butt	CRUD pages (Add, Update, Delete), CSV Upload/Export logic	Usman-0006
Muhammad Abdullah	Advanced Search filtering, Plotly Dashboards, Duplicate Detection	sesquad2025-maker

12. Challenges Faced and Solutions

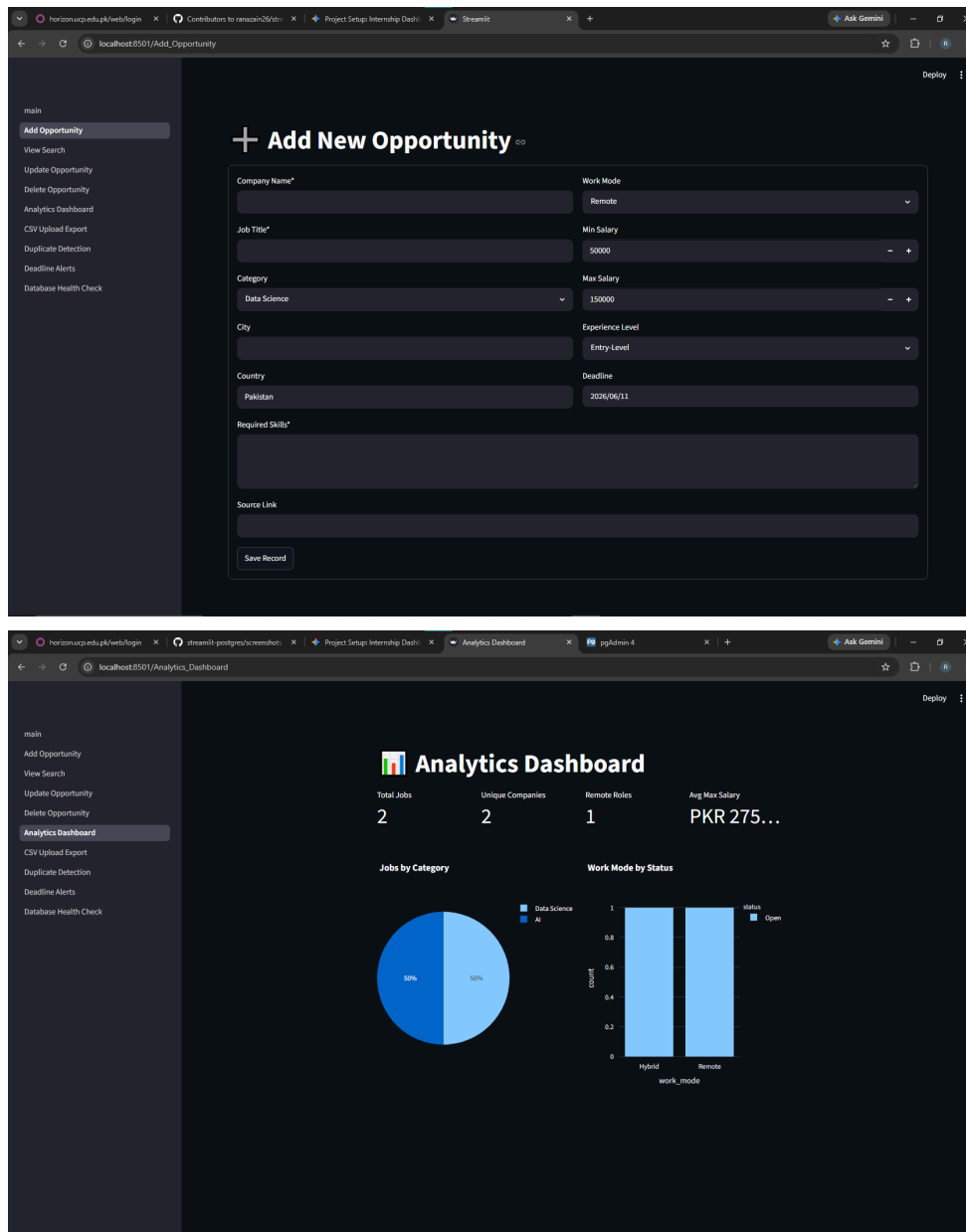
During development, the team encountered and resolved several complex infrastructure issues:

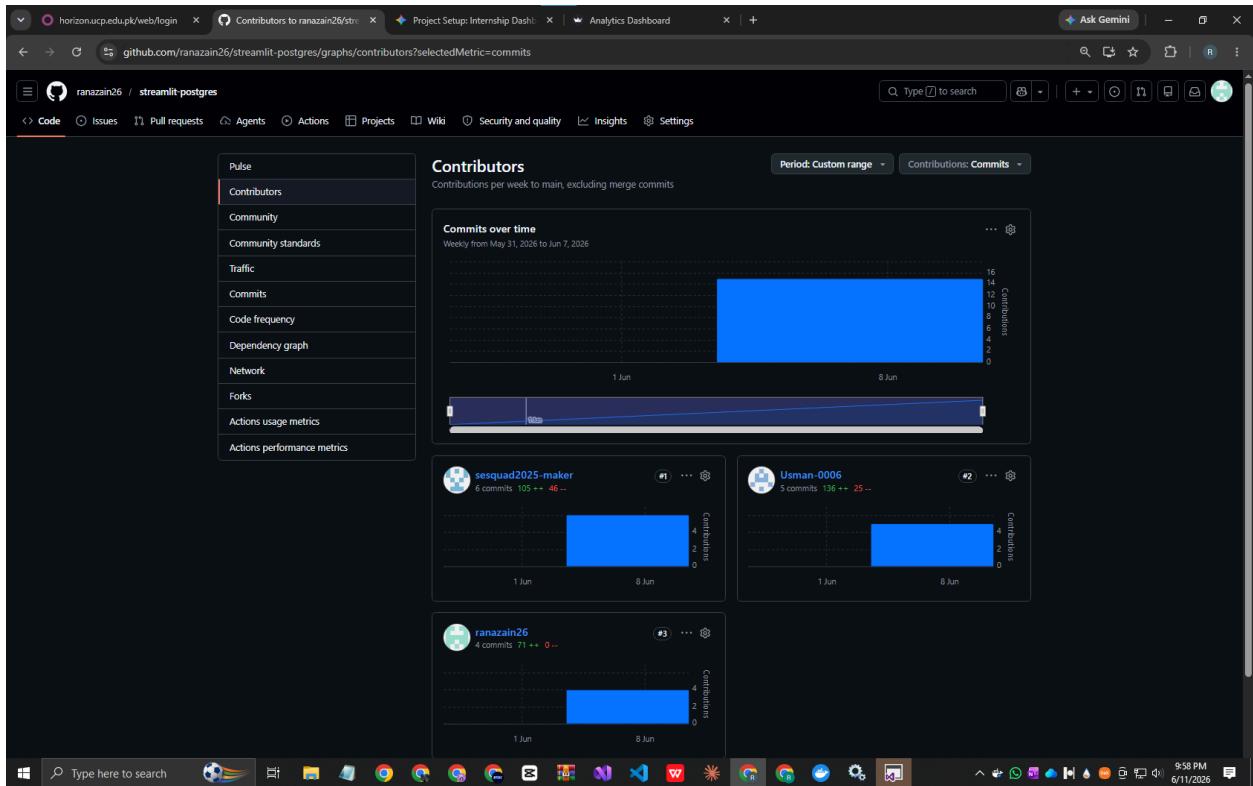
- Host Port Allocation Conflict: * Issue:** Docker failed to start with the error Bind for 0.0.0.0:5432 failed: port is already allocated. This occurred because a local Windows installation of PostgreSQL was already occupying port 5432.
 - Solution:** We modified the docker-compose.yml to map the external port to 5433 ("5433:5432"), allowing the container to run without host interference while maintaining the internal 5432 Docker network routing.
- PostgreSQL Version 18 Data Volume Crash:**
 - Issue:** Using the postgres:latest image caused the database to crash immediately. The logs revealed a file structure incompatibility because the latest tag had recently upgraded to version 18, which conflicts with standard /var/lib/postgresql/data volume mappings.
 - Solution:** We pinned the image version to the highly stable postgres:16 and executed docker compose down -v to permanently delete the corrupted volume and allow a clean initialization.
- Internal Network Resolution Failure:**
 - Issue:** Streamlit threw an Operational Error stating could not translate host name "postgres_db" to address.
 - Solution:** This was caused by the database crashing during network creation. By executing a clean teardown (docker compose down) and a rebuilt deployment (docker compose up -d --build), the internal Docker bridge was correctly re-established.

13. Conclusion

This project successfully demonstrated the end-to-end lifecycle of building a reproducible data application. By integrating Streamlit, PostgreSQL, and Docker Compose, the team delivered a fully functional tracking system that meets all departmental requirements. The process reinforced advanced software engineering practices, specifically containerized networking, relational database management, and collaborative version control using Git.

14. All Screenshots:





Commits on Jun 11, 2026

resolved issues	73d736
Administrator committed 31 minutes ago	
Update README.md	6c8de31
sesquad2025-maker authored 1 hour ago	
Update .env.example	fb3bd9a
sesquad2025-maker authored 1 hour ago	
Create format_currency function in utils.py	627cce
sesquad2025-maker authored 1 hour ago	
Add authentication role checks for admin access	a5da4d
ranazain26 authored 1 hour ago	
Add Database Health Check page with Streamlit	2cac1f4
ranazain26 authored 1 hour ago	
Add Deadline Priority Alerts page	2a8b8d6
ranazain26 authored 1 hour ago	
Add delete_opportunity_by_id function to queries.py	23d2ac
ranazain26 authored 1 hour ago	
Add duplicate detection functionality	bef4b6d
sesquad2025-maker authored 1 hour ago	
Add Analytics Dashboard with job metrics and charts	85bc7b6
sesquad2025-maker authored 1 hour ago	
Add search functionality to opportunities page	2b6d314
sesquad2025-maker authored 1 hour ago	
Update 6_CSV_Upload_Export.py	5d67a4d
Usman-0006 authored 2 hours ago	
Implement opportunity deletion	cccc5d4
Usman-0006 authored 2 hours ago	
Implement opportunity updates	50ff1e1
Usman-0006 authored 2 hours ago	
Replace ML model code with Streamlit form for opportunities	11ae763
Usman-0006 authored 2 hours ago	
Update 1_Add_Opportunity.py	80d85dc
Usman-0006 authored 2 hours ago	
Initial project structure	2c479b6
Administrator committed 2 hours ago	
Initial project structure	236ca0e
Administrator committed 2 hours ago	

